

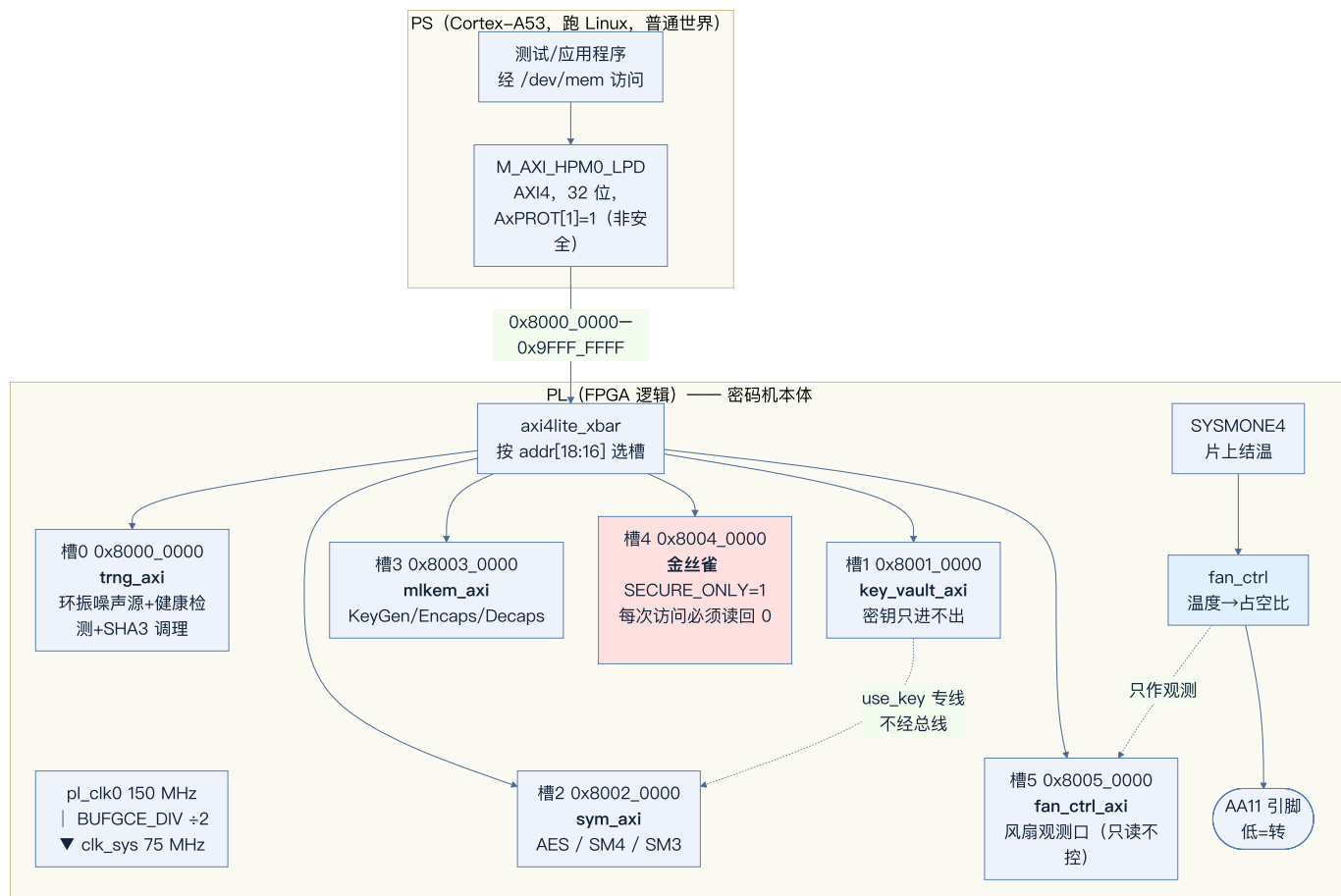
后量子密码机原型 —— 设计与验证参考

ZU3EG / XCZU3EG · 分支 zu3eg-fpga-crypto

- [架构](#)
 - [拓扑](#)
 - [地址映射](#)
 - [时钟与复位](#)
 - [密码核](#)
 - [密钥仓与防火墙](#)
 - [熵源](#)
 - [软件栈](#)
 - [主机侧密钥层次](#)
 - [集成注意事项](#)
- [API](#)
 - [分层](#)
 - [SDF 风格接口](#)
 - [与 GM/T 0018 的对应](#)
 - [PKCS#11 v3.2](#)
 - [私钥在哪里](#)
- [寄存器参考](#)
 - [槽位映射](#)
 - [trng_axi — 槽位 0](#)
 - [key_vault_axi — 槽位 1](#)
 - [sym_axi — 槽位 2](#)
 - [mlkem_axi — 槽位 3](#)
 - [pqc_accel_axi — 仿真与主机路径](#)
 - [通用约定](#)
- [安全模型](#)
 - [边界](#)
 - [威胁模型](#)
 - [已经证明了什么](#)
 - [哪些没有防护](#)
 - [局限](#)
- [测试](#)
 - [概览](#)
 - [原则](#)
 - [RTL 验证](#)

- [实现流程断言](#)
- [真硅结果](#)
- [熵](#)
- [主机软件测试](#)
- [复现](#)

本文由仓库 [docs/](#) 下的中文文档拼合生成，内容与仓库保持一致。文中所有测量数字均来自真实硬件运行，不是仿真、不是估算。原始日志见仓库 [board/logs/](#)。



总体架构

架构

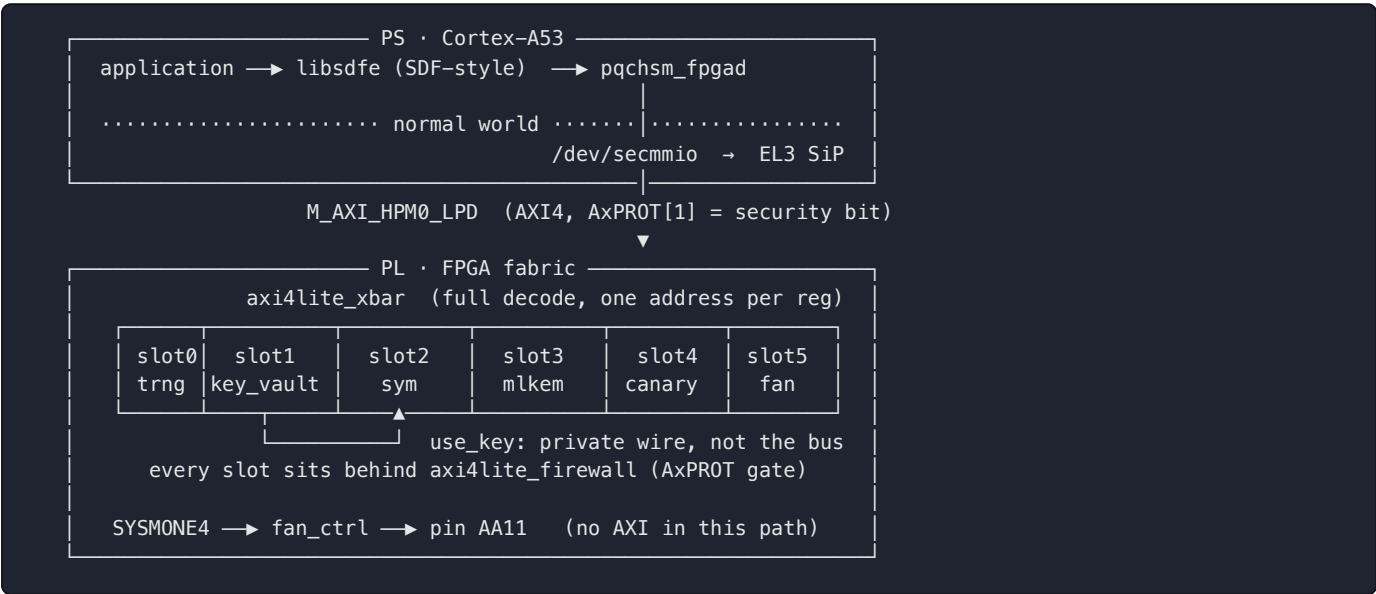
模块是怎么搭起来的，以及边界为什么落在这些地方。寄存器级细节见 [REGISTERS.zh-CN.md](#)；边界声称保证了什么，见 [SECURITY.zh-CN.md](#)。

- [拓扑](#)
- [地址映射](#)
- [时钟与复位](#)
- [密码核](#)
- [密钥仓与防火墙](#)
- [熵源](#)

- [软件栈](#)
- [主机侧密钥层次](#)
- [集成注意事项](#)

拓扑

器件是 Zynq UltraScale+ MPSoC：硬核 ARM（PS）与 FPGA 逻辑阵列（PL）同处一片 裸片，由 AXI 相连。所有密码学部分都在 PL 里。PS 跑 Linux 并下发命令，它自己 不持有任何密钥材料。



有两条通路是刻意绕开总线的：

- **use_key** —— 密钥仓通过私有连线把密钥交给对称密码核。密钥材料在总线上 根本不存在可读通路；见[密钥仓与防火墙](#)。
- **风扇控制** —— 结温经 DRP 从 PL 自带的 SYSMONE4 读出，直接驱动风扇引脚。Linux 不工作时散热也必须工作：上电头几秒、U-Boot 停在提示符时、内核挂死时。AXI 槽位 5 的接口只用于观测；把它删掉，风扇照样转。

地址映射

基址 `0x8000_0000`，槽位由 `addr[18:16]` 选中，每槽 64 KB。

地址	从机	SECURE_ONLY	用途
<code>0x8000_0000</code>	<code>trng_axi</code>	0	熵源。读 <code>0x08</code> 弹出一个 32 位字
<code>0x8001_0000</code>	<code>key_vault_axi</code>	0	密钥仓。只有写入通路；没有读出通路
<code>0x8002_0000</code>	<code>sym_axi</code>	0	AES-128/256、SM4、SM3
<code>0x8003_0000</code>	<code>mlkem_axi</code>	0	ML-KEM 512/768/1024，参数集由 <code>MODE[3:2]</code> 决定

地址	从机	SECURE_ONLY	用途
0x8004_0000	哨兵 (key_vault_axi)	1	与槽位 1 同一个模块，只有这一个参数不同
0x8005_0000	fan_ctrl_axi	0	风扇温度 / 占空比观测

译码是一一对应的：一个寄存器有且只有一个地址能够到它。axi4lite_xbar 检查孔径高位、槽位号、槽内偏移的高位以及 32 位对齐；四项必须全部成立，否则事务就地被拒——读回 0、写被丢弃——从机什么都看不到。

这份严格不是做样子。PS 侧没有任何保护单元覆盖这个窗口——UG1085 v2.5 把 0x8000_0000 排在 XPPU 的孔径表之外，也不在 FPD_XMPU 的通路上——所以译码加上 PL 里的防火墙，是这条路径上唯一的强制层。镜像地址等于给这一道防线开了成千上万个等价入口，每一个都返回 OKAY 且不留痕迹。test_xbar 逐字扫完整个 64 KB 槽位：恰好 0x00 - 0xFC 这 64 个地址可达，其余 16,320 个全部读回 0。

四个功能从机现在全是 SECURE_ONLY=1，以及这一步是怎么走到的。Linux 跑在普通世界，它经 /dev/mem 发出的每一笔事务都带 AxPROT[1] = 1。四个全设成 1 之后，Linux 一个密码核寄存器都读不到。

第一版有意没这么做：四个功能从机设成 SECURE_ONLY=0，让 Linux 能直接跑 KAT，另在槽位 4 上放一个只有这一个参数不同的哨兵实例来演示门控。那一版证明了两件真事（算法在真硅上对；AxPROT 门控对着真实的非安全主设备确实生效），但留了个缺口：被证明受保护的是那个空壳哨兵，不是密码核本身。

现在这一版把缺口补上了。四个全是 SECURE_ONLY=1，整套 KAT 改由安全世界驱动——打过补丁的 BL31 提供一个受限的安全 MMIO SIP，白名单正好覆盖这几个核的合法寄存器偏移

(boot/atf/patch_atf_secmmio.py)，普通世界的测试程序把每一笔核访问都经 SMC 转过去。哨兵留在槽位 4 作同参数对照。PQC_DEV_OPEN=1 仍然能出原来那份开放构建 (zu3eg_hsm_dev.bit)，用于调试。

时钟与复位

```
pl_clk0 (150 MHz, from PS)
└─ BUFGCE_DIV ÷2 → clk_sys (75 MHz) → all crypto cores + fan + xbar
                                     └─ PS maxihpm0_lpd_aclk

pl_resetr0 (async) → 2-stage synchroniser ┌─ & → rst_n
fabric power-on reset (count to 255 after GSR)└─
```

- 75 MHz，不是 150。最慢的核 mlkem_decaps 在 out-of-context 下收敛于 108.5 MHz。75 MHz 给它留了 45 % 余量，而这正是全部接起来之后时钟树与总线集成所吃掉的量。
- 用 BUFGCE_DIV，不用 MMCM。MMCM 有锁定要等，有复位次序要摆对——多出来的状态就是多出来的出错可能。纯分频器从上电那一刻起就是对的。
- 另外还有一路由逻辑阵列自己产生的上电复位。配置完成后 FPGA 的每个触发器都是 0 (GSR)，所以那个计数器必然从 0 开始。有了它，即使运行时重配置之后 pl_resetr0 没有干净释放，设计照样能起来——少一个只在真硅上才现形的依赖。

密码核

全程是可推断的纯 Verilog-2001：任何密码模块里都没有例化厂商原语，所以同一份源码不作改动即可面向 Xilinx、Intel 或 Lattice。（厂商原语只出现在板级顶层，而 hardware/tb/lint/vendor_stubs.v 提供了空壳，使顶层仍能通过 lint，而不是被排除在 lint 之外。）

目录	模块数	内容
<code>hardware/rtl/mlkem/</code>	27	NTT、基乘、Montgomery / Barrett 约简、采样、压缩、编码 / 解码，以及完整的 KeyGen / Encaps / Decaps 核
<code>hardware/rtl/mldsa/</code>	13	ML-DSA 算子——NTT、舍入、提示、采样。已验证，但未串成完整的核
<code>hardware/rtl/keccak/</code>	2	<code>keccak_f1600</code> 置换与 <code>sha3_core</code> 海绵（G / PRF / XOF / H 共用一个实例）
<code>hardware/rtl/sym/</code>	6	AES-128/256、SM4、SM3
<code>hardware/rtl/trng/</code>	6	环形振荡器源、RCT / APT 健康测试、SHA-3 后处理、可擦除 FIFO、AXI 封装
<code>hardware/rtl/bus/</code>	8	AXI4-Lite 防火墙、密钥仓、各核的 AXI 封装
<code>hardware/rtl/board/</code>	2	<code>axi4lite_xbar</code> 交叉开关与板级顶层 <code>zu3eg_hsm_top</code>
<code>hardware/platform/fan_ctrl/</code>	4	风扇控制——不是密码部分，单独放一棵树；只共用时钟与复位

算法清单、参数集与验证状态列表见 [reference/security-policy.zh-CN.md](#)。

`sha3_core` 用一个实例服务 ML-KEM 里全部四种 Keccak 用途。Decaps 的重加密 步骤直接例化 Encaps 核——这不是走捷径，FIPS 203 §7.3 对该运算的定义就是如此。

恒定时间只在一处不容商量：Decaps 的隐式拒绝比较会比完**每一个**字节再做选择，绝不在第一个不同处提前返回。

密钥仓与防火墙

不变量不是”密钥很难被读到”，而是**根本没有通路**。

密钥经总线写入 `key_vault`，只经 `use_key` 连线出去、进入对称密码核。RTL 里 不存在对密钥寄存器的总线侧读取，所以没有任何软件——包括装载这把密钥的那个 守护进程——能把它取回来。相关规则：

- **装了一半的密钥不算密钥**。只有 `COMMIT` 之后密钥才可用；写了一部分的 用不了。
- **擦除只要一个周期**。整个密钥仓在一个时钟内清空，而不是逐字扫过，所以 不存在半把密钥还在的窗口。
- **防拆是单向的**。防拆输入能擦除，永远不能反擦除。
- **违规会留痕**，所以被拒的访问与从未发生过的访问可以区分开。

`axi4lite_firewall` 为每个从机做门控。`SECURE_ONLY=1` 时它要求 `AxPROT[1] == 0`；否则**读回 0、写被丢弃**，且不产生总线错误（RAZ/WI）。有两个细节要紧：

- **被拒的读绝不弹出 FIFO**。否则一次拿不到数据的非安全读仍然会消耗随机字——那就成了普通世界抽干熵池的手段。改 RAZ/WI 之后这一条**更要紧了**：响应码不再区分放行与拒绝，“FIFO 没动”成了唯一能证明”事务没往下走”的观测点。

- 是 RAZ/WI，不是 DECERR —— 这一条以前写的正好相反。原文是“是 DECERR，不是 OKAY 加返回零：拒绝必须是可见的”。它被一个更坏的失败方式推翻了：被拒的写是 posted 的，它的 DECERR 以 SError 回来，内核只能 panic —— 一个打错的地址，一次断电。这一下板子真挨过，而且是内核自己的 GPIO 探测打的，不是攻击者。

拒绝仍然可见，只是不再经由响应码：每个核的 `VERSION` 都是非零常量，所以读到 `0` 本身就是那个信号；违规计数器（只有安全世界读得到）留下审计痕迹。真正让出去的是“打错地址的可诊断性”，换来的是任何用户态程序都搞不崩这块板。

熵源

```
ring-osc array —sample/decimate—┐
                                ├── continuous health tests (RCT + APT)
                                └── Keccak sponge conditioning → FIFO → AXI
```

八个长度不同的环形振荡器（13/15/.../27 级），两级同步器，然后抽取。

- 后处理用的是 Keccak 海绵，不是自制白化器。SP 800-90B §3.1.5.1.2 列出了经审定的后处理构造；用其中之一，就可以直接把输出熵计为 `min(output length, input entropy)`，而自创的白化器得自己另行论证。它还复用了 `keccak_f1600`，不额外占面积。
- 健康测试消费的是抽取之后的流——与后处理器消费的是同一条流。测的东西不是用的东西，什么也证明不了。
- 报警时会清空熵池。报警 → `ready` 拉低 → FIFO 擦除 → 后处理器与海绵 状态复位 → 重跑启动测试。标准只要求停止输出；但报警意味着源可能已经劣化了一段时间，所以留着池子比丢掉更危险。
- 门限来自实测熵，不是假设。见 [TESTING.zh-CN.md](#)。

软件栈

PKCS#11 v3.2 shared library src/p11/		SDF-style library + daemon service/	
slot manager src/slot/	keystore src/store/	backup / recovery src/backup/	audit src/audit/
pqc_backend_t		include/pqchsm/pqc.h	
accel_transport_t		include/pqchsm/accel.h	
stub Verilator RTL AXI /dev/mem + mmap			

每一层只依赖它下面那一层的接口，`pqc_backend_t` 虚表以上的一切都只操作不透明句柄。`include/pqchsm/` 里声明的函数没有一个返回私钥材料：密钥生成返回句柄，签名接收句柄与消息，解封装接收句柄与密文。句柄编码了槽位以及该槽位的世代计数，所以销毁一个对象会让先前为它发出的所有句柄失效，而不是悄悄把它们重新绑定到一把新密钥上。

硬件接缝先于任何 RTL 存在。`accel.h` 把寄存器契约定死——`CTRL`、`STATUS`、`MODE`、`PARAM`、`IN_LEN`、`OUT_LEN`、`ERRCODE`——所以接口上的错误先在软件里暴露；有四种 transport 以完全相同的方式实现它：软件桩、Verilator 仿真的 RTL、经真实 AXI4-Lite 与 AXI4-Stream 驱动的同一批 RTL，以及板上的 `/dev/mem` + `mmap`。桩与仿真 RTL 经由它必须给出逐字节相同的结果，这一点由 `tests/unit/test_accel.c` 断言。

`accel_shake()` 用 C 实现海绵——填充、rate、吸收、挤出——只在做置换时调用 transport。只做置换的核更小，而且 SHAKE128、SHAKE256、SHA3-256 与 SHA3-512 一律照用。

哪些是在真硅上跑过的。ACVP 结果、对称算法向量、熵采集与边界证明，来自 `board/src/` 下的独立程序，以及它们之上的 `service/`。`accel_transport_t` 之上的 `src/` 主机栈没有对着真实可编程逻辑跑过；把它接起来是另一件事。

主机侧密钥层次

```
KDR (device-bound root, 32 bytes – currently a stub)
└─ KEK      wraps key material at rest; cannot leave the device
RMK (recovery master key, randomly generated)
└─ BEK      wraps backup blobs; portable to a replacement device
    └─ Shamir M-of-N shares over GF(256)
```

之所以有两把包裹密钥，是因为目的相互冲突：受设备绑定密钥保护的材料很安全，但设备一坏就找不回来；受可携带密钥保护的材料能找回来，但其保护强度不超过分片保管的强度。每个槽位都带一个策略位，决定它的密钥是否参与备份，所以设备身份密钥可以被有意做成不可恢复。

一个密钥库文件由逐槽位的 blob 加上整文件 MAC 构成：

```
slot blob := plaintext metadata || AES-256-GCM(KEK, aad = metadata, key material || PIN material)
file      := header || slot blob × N || KMAC256(file)
```

元数据只做认证而不加密，所以不解包就能读到某个槽位的算法、用途与策略，同时仍然可发现篡改。写是原子的——临时文件 → `fsync` → `rename` → `fsync(dir)`——所以崩溃之后留下的要么是旧文件，要么是新文件。

每一次状态迁移都追加一条审计记录，其 SHA3-256 哈希覆盖前一条记录的哈希。纯链条挡不住把整个文件重写一遍的攻击者，所以链头由 ML-DSA 设备身份密钥签名，并锚定在设备之外。

密钥注入在不暴露密钥的前提下完成灌装：灌装主机对设备自己的 ML-KEM 公钥做封装，并用得到的共享秘密包裹一个种子。FIPS 203 与 204 的密钥完全由种子决定，所以这样搬动的敏感字节更少，而且设备可以自己验证展开结果。

集成注意事项

`hardware/rtl/board/zu3eg_hsm_top.v` 在一个文件里把所有东西接起来。其中有三件事是在真实硬件上付过代价的：

`M_AXI_HPM0_LPD` 是 AXI4，不是 AXI4-Lite。当成 Lite 处理，`bid`、`rid` 与 `rlast`——都由 PL 驱动、都要回给 PS——就悬空了。PS 会永远等 `rlast`，CPU 就挂在那一条 load 指令上。综合、布局布线、时序与位流生成全都干净，板子甚至报告“运行中”。AXI4-Lite 就是把突发长度固定为 1 的 AXI4，所以 `rlast = rvalid`，`bid` / `rid` 回显 `awid` / `arid`。实现流程现在带了一条综合后断言来盯这件事。

声明顺序不是风格问题。在 Vivado 里引用一个后面才声明的 `wire` 不会报错——它会在那个端口上悄悄新建一根同名、无驱动的网线。Icarus 则直接拒绝同样的代码，这正是 lint 流程要有厂商空壳的原因：把板级顶层排除在 lint 之外，就等于关掉了唯一能自动抓到这件事的检查。

风扇安全一律偏向多吹风。最小占空比是 25 % 而不是 0；≥80 °C 强制 100 %， 直到降到 74 °C 才释放；温度陈旧强制 100 %，因为温度未知时唯一安全的假设就是 它很高。第四条规则是一次上板之后加的：读数一个比特都不变，同样强制 100 %。 第一版 SYSMON 配置错得很巧妙——DRP 照样应答，寄存器里也照样是看着合理的 32.5 °C——只是 ADC 根本没在转换。“读不到”这个条件从未成立，所以陈旧规则触发 不了。真实的结温总在抖，所以完美的稳定本身就是故障信号。

API

同一套硬件之上提供两个前端：一个是 **SDF 风格 C 库**（`service/`），经安全世界 与 FPGA 核对话；另一个是主机软件栈之上的 **PKCS#11 v3.2 模块**（`src/p11/`）。

- [分层](#)
- [SDF 风格接口](#)
- [与 GM/T 0018 的对应](#)
- [PKCS#11 v3.2](#)
- [私钥在哪里](#)

分层

application	business code, standard interfaces only
libsdf / pqchsm-pkcs11.so	standard API → internal wire stateless, safe across processes
pqchsm_fpgad	sessions, handles, ACL, serialisation
/dev/secmmio	misc device turning requests into SMCs. NOT a trust boundary – a courier
BL31 SiP (EL3) AxPROT[1] = 0	whitelist, and the only thing that can issue a secure transaction ← boundary
FPGA cores SECURE_ONLY=1	AXI firewall → trng / key_vault / sym / mlkem. Non-secure access refused at the bus

每一层为什么必须存在：

层	解决什么问题	没有它会怎样
标准库	应用换模块不必改代码	每个应用都得自己拼装线协议
守护进程	多个进程共享同一块硬件：会话、句柄、串行化	两个进程同时驱动 <code>mlkem_axi</code> ，互相踩对方的寄存器
内核节点	用户空间发不了 SMC	命令逻辑就得搬进内核
EL3 SiP	普通世界的事务总是带着 <code>AxPROT[1] = 1</code> 而被拒	在 <code>SECURE_ONLY=1</code> 的构建里，这些核根本没法用
PL 防火墙	门控本身	没有边界——而且 PS 侧没有任何东西能顶替它

守护进程是最常被认为可以砍掉的一层。它砍不得：三个核都是有状态的序列——写寄存器、启动、轮询、读结果——两个进程在同一个核上交错，产出的结果彼此错位、单看却都说得通。串行化需要一个单点，而这个单点必须记住会话。

SDF 风格接口

`service/sdfe.h`。前缀是 `SDFE_`（“SDF Extended”），不是 `SDF_`：命名沿用 GM/T 0018 的惯例，但国密标准尚未定义 ML-KEM 接口，因此这些名字不冒充标准函数名。

这个库不做任何密码运算。它把一次调用翻译成一条发给 `pqchsm_fpgad` 的请求。它是无状态的；不同进程可以各自链接它、各自开会话，永远看不到对方的句柄。

```
/* 设备与会话 */
int SD FE_OpenDevice(SD FE_HANDLE *phDev);
int SD FE_CloseDevice(SD FE_HANDLE hDev);
int SD FE_OpenSession(SD FE_HANDLE hDev, SD FE_HANDLE *phSession);
int SD FE_CloseSession(SD FE_HANDLE hSession);
int SD FE_GetDeviceInfo(SD FE_HANDLE hSession, char *buf, size_t cap);

/* 随机数——来自 PL 里的环形振荡器源，不是软件 PRNG */
int SD FE_GenerateRandom(SD FE_HANDLE hSession, uint32_t len, uint8_t *out);

/* ML-KEM——dk 留在守护进程；应用拿到的是句柄 */
int SD FE_GenerateKeyPair_MLKEM(SD FE_HANDLE hSession, uint32_t pset,
                                uint8_t *ek, uint32_t *ek_len,
                                uint32_t *key_handle);
int SD FE_Encapsulate_MLKEM(SD FE_HANDLE hSession, uint32_t pset,
                            const uint8_t *ek, uint32_t ek_len,
                            uint8_t *ss, uint32_t *ss_len,
                            uint8_t *ct, uint32_t *ct_len);
int SD FE_Decapsulate_MLKEM(SD FE_HANDLE hSession, uint32_t key_handle,
                            const uint8_t *ct, uint32_t ct_len,
                            uint8_t *ss, uint32_t *ss_len);

/* 对称——密钥写入 key_vault，此后按槽位使用 */
int SD FE_ImportKey(SD FE_HANDLE hSession, uint32_t slot,
                    const uint8_t *key, uint32_t key_len);
int SD FE_Encrypt(SD FE_HANDLE hSession, uint32_t alg, uint32_t slot,
                  const uint8_t *in, uint8_t *out); /* 单个 16 字节分组 */
int SD FE_Decrypt(SD FE_HANDLE hSession, uint32_t alg, uint32_t slot,
                  const uint8_t *in, uint8_t *out);

const char *SD FE_StrError(int rv);
```

返回码沿用 SDF 惯例：成功为 `SDR_OK` (0)，否则为 `SDR_BASE + n` —— `SDR_OPENDEVICE`、`SDR_COMMFAIL`、`SDR_INARGERR`、`SDR_KEYNOTEXIST`、`SDR_HARDFAIL`、`SDR_UNKNOWERR`。

参数集为 `SD FE_MLKEM_512` / `_768` / `_1024`；对称算法为 `SD FE_ALG_AES128` / `_AES256` / `_SM4`，与 `sym_axi` 的 `ALG` 字段对应。

`service/sdf_demo.c` 是一个完整示例，其中不含任何硬件细节——没有寄存器、没有 `/dev/mem`、没有 SMC——并且只链接 `libsdfe`，不链接密码库。它自己算不出任何东西；它打印出的每一个正确结果都来自 FPGA。见 [USAGE.zh-CN.md](#)。

与 GM/T 0018 的对应

SDF 函数	对应到	状态
<code>SDF_OpenDevice</code> / <code>CloseDevice</code>	连接/断开守护进程套接字	已具备
<code>SDF_OpenSession</code> / <code>CloseSession</code>	<code>CMD_SESSION_OPEN</code> / <code>_CLOSE</code>	已实现
<code>SDF_GetDeviceInfo</code>	静态能力表 + 槽位信息	已具备
<code>SDF_GenerateRandom</code>	经 EL3 读 <code>trng_axi</code> 的 <code>RDATA</code>	硬件
<code>SDF_Encrypt</code> / <code>SDF_Decrypt</code> (SM4)	<code>sym_axi</code> + <code>key_vault</code> 槽位	硬件
<code>SDF_HashInit/Update/Final</code> (SM3)	<code>sym_axi</code> 的 SM3 通路	硬件
<code>SDF_GenerateKeyPair_ECC</code> (SM2)	—	✗ 没有 SM2 核
<code>SDF_InternalSign_ECC</code> / <code>InternalVerify_ECC</code>	—	✗ 没有 SM2 核
<code>SDF_ImportKeyWithISK_ECC</code> 及密钥 交换一族	依赖 SM2	✗ 没有 SM2 核
<code>SDF_ExportSignPublicKey_ECC</code>	公钥可以出去；私钥永不	结构上成立

SDF 中内部密钥的概念天然对应 `key_vault`：`SDF_InternalSign` 接受的是设备内部 密钥索引 (`uiISKIndex`)，从不携带私钥，这正是“按槽位或句柄引用，永不离开硬件”。

把差距说清楚：本设计覆盖的是 SDF 的对称、杂凑与随机数服务。非对称一族（SM2/ECC）在这里没有硬件，而对国密送检来说，这是一块相当大的缺口。

ML-KEM 目前还没有国标接口，因此有两条路可走。本项目把 PKCS#11 v3.2 的 PQC 机制当作主路径——国际标准已经定稿，头文件已随仓库引入，`C_Encapsulate` / `C_DecapsulateKey` 已实现——同时把 `SDFE_*` 这套名字保留为同一批线上命令之上的一层薄封装，这样等国标真正落地时，改动只局限在那一层。

PKCS#11 v3.2

`src/p11/p11_module.c` 实现了 44 个 `C_` 函数。用 `cmake --build build --target pqchsm-p11` 构建。

```
CK_INTERFACE *iface;
C_GetInterface(NULL, NULL, &iface, 0); /* v3.2 的机制在这里 */
```

机制	由什么支撑	状态
<code>CKM_ML_KEM</code> (0x17)	<code>mlkem_axi</code> ，三个参数集全部支持	硬件
<code>CKM_AES_ECB</code> / <code>CKM_AES_CBC</code>	<code>sym_axi</code> 的 AES-128/256	硬件
<code>CKM_SHA3_256</code>	<code>sha3_core</code> （目前只能经 ML-KEM 到达）	需要一条专用通路
<code>CKM_ML_DSA</code> (0x1D)	ML-DSA 算子，未串成核	✗ 仅软件

机制	由什么支撑	状态
SM4 / SM3	不存在标准机制码	厂商自定义码

`C_GenerateKeyPair` → `CMD_GENERATE` → `mlkem_axi` KeyGen; `C_DecapsulateKey` → `CMD_DECAPS` → `mlkem_axi` Decaps。

对象模型紧贴 PKCS#11，因为一一对应可以省掉一层阻抗匹配。每个槽位最多持有一对 密钥，因而恰好呈现两个对象，两者的句柄可以互相推出；不需要单独的对象表。访问 控制基于角色（`SO`、`USER`），在槽位管理器而不是 PKCS#11 层执行，因此命令行与 守护进程得到的是完全一致的约束。

已知偏差：`CKM_HASH_ML_DSA_*` 未实现，因此多段签名会把整条消息缓存下来，而不是 流式计算摘要。上层 provider 框架（JCA、PyCryptodome 之流）尚不支持这些机制；`demo/` 里的演示因此直接使用低层绑定，`demo/java/SunP11Probe.java` 演示了为什么。

私钥在哪里

有两种情形，不能揉成一句话。

密钥	存在哪里	会离开硬件吗？
对称（AES / SM4）	<code>key_vault</code> 寄存器阵列	不会。RTL 里没有总线侧的读出路径。演示程序在导入后擦掉自己那份副本，此后没有任何人——包括守护进程——能把它取回来
ML-KEM <code>dk</code>	目前由 <code>KeyGen</code> 返回；由守护进程持有	⚠ 会，它会离开硬件——但不会离开接口。应用手里始终只有一个句柄

第二行是当前状态，不能简化成”私钥永不离开硬件”。要让那句话成立，需要在 `mlkem_axi` 内部实现私钥存储与按句柄使用，那是产品级的工作，不是原型级的。见 [SECURITY.zh-CN.md](#)。

寄存器参考

⚠ 被拒的访问长什么样：读回 0，不是报错

默认位流（`zu3eg_hsm.bit`）对普通世界是零可达的。四个功能从机都按 `SECURE_ONLY=1` 综合，而 Linux 发出的每一笔事务 `AxPROT[1]` 恒为 1，因此在总线上一律被拒。只有安全世界（EL3，经 BL31 的 SiP）驱动得了它们。

被拒的访问走 RAZ/WI：读回 0、写被丢弃，响应一律 OKAY，总线上不产生任何错误。所以：

- 判断一次访问有没有被拒，看数据不看错误码。各核的 `VERSION` 都是 `0x0001_0000`，读到 0 就是被拒了（或者位流没载）。
- 随便写，写不坏。任何用户态程序、任何驱动、任何手滑的地址都不会让内核收到总线错误。这一条在仿真里已经确立（`test_xbar` / `test_firewall` / `test_trng_axi`）。`board/src/hsm_nocrash.c` 是它在真硅上的对应物——九类地址各读写两千次——但还没上板跑过，见 [TESTING.zh-CN.md](#)。
- 代价是走错地址变安静了。打错一个地址不会报错，而是读回 0。补偿是违规计数器（各核 `VIOL`、译码器的 `XBAR_VIOL`），但它们只有安全世界读得到——普通世界读那两个地址同样是 0。

为什么不回 DECERR

DECERR 才是“正确”的 AXI 语义，第一版也是这么做的。换掉它是因为一个不对称：被拒的读是同步中止，Linux 转成 `SIGBUS`，程序接得住；被拒的写是 `posted` 的，错误过一会儿才以 `SError` 回来，不属于任何一条指令，内核只能 panic——代价是一次断电，而这块板断电会清掉 `CSU_MULTI_BOOT`。

而触发它不需要恶意：实测过一次，设备树里留着厂家 PL 的 GPIO 节点，换上密码位流后内核自己的 `xgpio_of_probe` 去探测一个不存在的外设，当场 panic。一个密码机不该有“用户态一个手滑就得断电”这种性质，所以改了。

安全性没有变化：写到不了从机、读拿不到密钥，与 DECERR 时逐字一样，变的只是总线不再报错。

`PQC_DEV_OPEN=1` 现在只为一件事存在

```
PQC_DEV_OPEN=1 vivado -mode batch -source hardware/syn/impl_bitstream.tcl
```

这份构建把功能从机设成 `SECURE_ONLY=0`（产物 `zu3eg_hsm_dev.bit`），用途是让普通世界能直接跑 KAT，方便调试。它不再是“避免写崩板”的必需品——那条限制已经没有了。它是调试形态，不是交付形态。

软件与设计中每一个 AXI 从机之间的契约。两侧都照着本文档编写，其中每一条都由 cocotb testbench 单独验证。

所有寄存器均为 32 位且按字对齐。每个从机都位于 `axi4lite_firewall` 之后；落在从机窗口之外的地址，以及对 `SECURE_ONLY=1` 实例的非安全访问，一律读回 0、写被丢弃（RAZ/WI），且无副作用、不产生总线错误。

- [槽位映射](#)
- [trng_axi](#)
- [key_vault_axi](#)
- [sym_axi](#)
- [mlkem_axi](#)

- [pgc_accel_axi](#)
- [通用约定](#)

槽位映射

基址 `0x8000_0000`，槽位取自 `addr[18:16]`，每槽 64 KB。译码为何是精确的，见 [ARCHITECTURE.zh-CN.md](#)。

槽位	地址	从机	SECURE_ONLY
0	<code>0x8000_0000</code>	<code>trng_axi</code>	0
1	<code>0x8001_0000</code>	<code>key_vault_axi</code>	0
2	<code>0x8002_0000</code>	<code>sym_axi</code>	0
3	<code>0x8003_0000</code>	<code>mlkem_axi</code>	0
4	<code>0x8004_0000</code>	哨兵 (<code>key_vault_axi</code>)	1
5	<code>0x8005_0000</code>	<code>fan_ctrl_axi</code>	0

`trng_axi` — 槽位 0

与其他从机不同，这是一个自由运行、始终在线的外设：没有命令 / 完成的循环，只有预热 → 就绪 → 取字，外加健康状态。RTL 见 `hardware/rtl/trng/trng_axi.v`，测试见 `hardware/tb/cocotb/test_trng_axi.py`。

偏移	名称	访问	说明
<code>0x00</code>	<code>CTRL</code>	RW	<code>[0]</code> ENABLE (电平)， <code>[1]</code> ZEROIZE (写 1 脉冲)， <code>[2]</code> CLEAR_ALARM (写 1 脉冲)
<code>0x04</code>	<code>STATUS</code>	R	见下
<code>0x08</code>	<code>RDATA</code>	R	每读一次弹出一个 32 位随机字
<code>0x0C</code>	<code>HEALTH</code>	R	<code>[31:16]</code> 当前 APT 窗口计数， <code>[15:0]</code> 当前 RCT 连长
<code>0x10</code>	<code>APT_INDEX</code>	R	本 APT 窗口内已处理的样本数
<code>0x14</code>	<code>STARTUP</code>	R	启动健康测试已通过的样本数
<code>0x18</code>	<code>BLOCKS</code>	R	后处理器已吸收的 rate 块数
<code>0x1C</code>	<code>WORDS</code>	R	已交付给软件的字数
<code>0x20</code>	<code>VERSION</code>	R	常量 <code>0x0001_0000</code>

偏移	名称	访问	说明
0x24	PARAM0	R	{DECIM, NUM_R0, RATE_LANES, OUT_LANES}, 各占一字节
0x28	PARAM1	R	{APT_CUTOFF[15:0], RCT_CUTOFF[15:0]}
0x2C	PARAM2	R	{STARTUP_SAMPLES[15:0], APT_WINDOW[15:0]}
0x30	RAW	R	原始噪声抽头, 仅 RAW_TAP=1 时有内容
0x34	DROPS	R	取样 FIFO 溢出计数
0x38	VIOL	R	{被拒的读[31:16], 被拒的写[15:0]}, 饱和

STATUS : [0] READY (启动测试已过、无报警、未在擦除), [1] DATA_VALID, [2] ALARM (锁存), [3] RCT_ALARM, [4] APT_ALARM, [5] STARTUP_DONE, [6] FIFO_WIPING, [7] ENABLED, [8] UNDERRUN (锁存)。

RDATA 读即弹出。 读一次消耗一个字。软件不得回读校验, 不得为确认而 重读该地址, 也不得在调试器里 dump 这段范围——每一次读都在消耗熵。AXI4-Lite 没有推测读, 所以一次读恰好等于一次消耗。

空读返回 0 并锁存 UNDERRUN。 返回 0 本身是危险的: 不查状态就读的驱动会把 0 当成随机数用。UNDERRUN 的存在是为了事后抓出这种驱动 bug; 它是锁存的, 只能经 ZEROIZE 或 CLEAR_ALARM 清除。

正确的驱动用法: 在每一次读 RDATA 之前读 STATUS 并确认 DATA_VALID。取完一批之后再读一次 STATUS, 确认 UNDERRUN 与 ALARM 都是清的——否则 整批丢弃。

ALARM 是锁存电平, 不是脉冲, 理由与别处的 DONE 相同: 软件在任意时刻 轮询, 单个周期会被错过。

PARAM1 里的门限由实测最小熵推导得出, 不是假设的; 见 TESTING.zh-CN.md。

key_vault_axi — 槽位 1

窗口 0x00 – 0x3F; 超出一律读回 0、写被丢弃。RTL 见 hardware/rtl/bus/key_vault_axi.v, 测试见 hardware/tb/cocotb/test_key_vault.py。

这个外设的存在, 是为了把”密钥不离开密码边界”变成电路的性质, 而不是一种 纪律。常见做法是把密钥放在内存里、靠软件不去读它, 这要求每一处访问点都记得 检查权限——漏掉一处就是一个无声的窟窿。这里在密钥寄存器与 s_axi_rdata 之间 干脆没有连线。软件可以写入密钥、把它标记为锁定、擦除它, 以及询问某个槽位 是否已装载。要使用密钥, 由 PL 里的核经 use 端口取走, 这束信号从不离开 芯片。

偏移	名称	访问	说明
0x00	VERSION	R	常量 0x0001_0000

偏移	名称	访问	说明
0x04	CTRL	W	[0] ZEROIZE——写 1 触发全局擦除，自清
0x08	STATUS	R	[0] READY, [1] TAMPER_LATCHED, [2] DENY (有操作被拒；锁存，由 ZEROIZE 清除)
0x0C	SLOT_SEL	RW	[2:0] 当前操作所用的槽位
0x10	KEY_IN	W	一个 32 位字；字索引自动前进。读回为 0
0x14	SLOT_CTRL	W	[0] BEGIN, [1] COMMIT, [2] LOCK, [3] ERASE (写 1 触发)
0x18	SLOT_STAT	R	[0] VALID, [1] LOCKED, [7:4] 迄今已写入的字数
0x1C	VALID_MAP	R	每槽位一比特
0x20	LOCK_MAP	R	每槽位一比特
0x24	ZERO_CNT	R	[7:0] 全局擦除次数，饱和计数
0x28	VIOL_CNT	R	{read violations[31:16], write violations[15:0]}，饱和计数
0x2C	VIOL_INFO	R	[7:0] 首次违规地址，[8] 为写操作，[9] NS 位，[10] 有效
0x30	PARAM0	R	{WORDS[15:8], SLOTS[7:0]}，供驱动自检

本表中没有任何地址会返回密钥材料。KEY_IN 只写，读回为 0——不是“被门控 成 0”，而是压根就没有接到密钥寄存器上。test_key_never_readable 用八个槽位 各装一把不同的密钥，逐字扫遍整个 256 字节地址空间，断言 64 个密钥字一个都不 出现，以此验证这一点。这个测试是咬人的：把任意一个密钥字接进读多路选择器， 它立刻就会报出泄漏的地址和值。

装载一把密钥：

```
SLOT_SEL  = n      select the slot
SLOT_CTRL = BEGIN  clear that slot's word index and VALID
KEY_IN    × 8      eight 32-bit words (256 bits)
SLOT_CTRL = COMMIT set VALID
SLOT_CTRL = LOCK   optional; afterwards it can be neither written nor erased
```

装了一半的密钥不算密钥：只有 COMMIT 才让槽位可用。擦除在一个周期内 完成——整个密钥仓，而不是逐字扫过——所以不存在半把密钥残存的窗口。防拆是 单向的：能擦除，永远不能反擦除。

sym_axi — 槽位 2

窗口 0x00 — 0x7F。RTL 见 hardware/rtl/bus/sym_axi.v。

本表里没有 KEY 寄存器，这是有意的。常见设计是软件把密钥写进密钥寄存器 供密码核读取——这既把密钥放上了总线，又让它躺在寄存器里，于是谁能读这段地址，谁就拿到了密钥。这里软件只写 KEY_SLOT，一个 3 位的槽位号；密钥本身经 key_vault 的 use 端口，走 256 根从不离开 PL 的连线过来。软件说的是”用槽位 3 里的密钥”，而不是”密钥是 0x...”。“所以”读密钥”这件事在总线上没有地址，而 不是有一个被门控的地址。

偏移	名称	访问	说明
0x00	VERSION	R	常量
0x04	CTRL	W	[0] ZEROIZE (写 1, 自清；擦除全部三个核)
0x08	STATUS	R	[0] BUSY, [1] DONE, [2] KEY_READY, [3] KV_VALID
0x0C	ALG	RW	[1:0] 0 = AES-128, 1 = AES-256, 2 = SM4, 3 = SM3; [2] DECRYPT
0x10	KEY_SLOT	RW	[2:0] 使用密钥仓的哪个槽位
0x14	CMD	W	[0] LOAD_KEY, [1] BLOCK, [2] HASH_START, [3] HASH_FINAL
0x18	HASH_IN	W	[7:0] 向 SM3 送入一个字节
0x1C	VIOL_CNT	R	{read[31:16], write[15:0]}
0x20 — 0x2C	DIN0..3	W	输入分组；DIN0 为最高的 32 位
0x30 — 0x3C	DOUT0..3	R	输出分组
0x40 — 0x5C	DIGEST0..7	R	SM3 摘要
0x70	PARAM0	R	支持算法位图

mlkem_axi — 槽位 3

RTL 见 hardware/rtl/bus/mlkem_axi.v，测试见 hardware/tb/cocotb/test_mlkem_axi.py。

偏移	名称	访问	说明
0x00	VERSION	R	常量 0x0001_0000
0x04	CTRL	W	START、ZEROIZE
0x08	STATUS	R	[0] BUSY, [1] DONE, [2] HASH_OK, [3] TAMPER, [4] WIPING, [5] PARAM_ERR
0x0C	MODE	RW	[1:0] 操作 (0 KeyGen, 1 Encaps, 2 Decaps), [3:2] 参数集 (0 = 512, 1 = 768, 2 = 1024)
0x10	IN_DATA	W	输入字节流; 写指针自动前进
0x14	IN_PTR	RW	输入写指针
0x18	OUT_DATA	R	输出字节流
0x1C	OUT_LEN	R	输出长度, 由硬件写入
0x20	OUT_RD	RW	输出读指针
0x24	VIOL_CNT	R	防火墙违规计数
0x28	PARAM0	R	能力字
0x2C	XBAR_VIOL	R	译码器违规 —— 没命中任何槽的访问次数

所有输入都经同一个缓冲区送入，顺序按标准的定义，因为三个核的输入形状完全不同，若每种形状配一组寄存器，就会得到三张互不相干的寄存器表和三份软件封送 代码：

```
KeyGen : d(32) || z(32)
Encaps : m(32) || ek(384k+32)
Decaps : dk(768k+96) || c(32·(du·k+dv))
```

那些必须并行呈现的 256 位量（**d**、**z**、**m**）由本模块从缓冲区头部取出，所以 软件永远不必知道哪些输入是流、哪些是并行端口。

所有长度都由 RTL 从 **pset** 算出。软件不上报长度，也就报不错——否则那会是一个悄悄产生错误答案的输入。

mode 与 **pset** 是 2 位，但只有 0/1/2 存在。值 3 不是另一种配置，它是一个不存在的东西。写入它会置 **PARAM_ERR**，**BUSY** 永不拉高，先前的 **DONE** / **OUT_LEN** 全部作废。读仍然允许，好让软件还能轮询。

Zeroize 是真的擦掉 BRAM。早先的版本只清 **in_ptr** / **out_len** / **out_rd** / **seed**。从软件看什么都读不到（**OUT_LEN = 0**），可上一次操作的 **dk** 每一个字节都还留在那两块 8 KB BRAM 里——这等于撕掉了目录 而正文还在。回去的路条条都通着：下一次操作只覆盖它用到的那段范围，旧私钥留在 尾部；位流回读或扫描链可以把它取出来；或者干脆把 **in_ptr** 推到旧区域再发起一次 操作。所以这里有一台真正的擦除机——由防拆或 zeroize 的上升沿触发，两块 BRAM 并行地逐地址写零，8192 个周期，全程 **WIPING = 1** 且拒绝输出读。

什么进得了缓冲区，什么进不了。ML-KEM 的 `dk` 确实会返回给软件——它是协议的私钥，由模块包裹后存到外部——所以私钥字节真的能从 `OUT_DATA` 读出来。这是 接口定义，不是泄漏。边界在于：任何中间值（ $s^{\wedge}e$ 、 $\hat{A}$ 、 $r^{\wedge}m'$ ，或重加密得到的 c' ）都不会进入缓冲区；它们只存在于核内部的 BRAM 里。这里 `DEBUG_BANK` 硬接为 0，多项式存储的读端口根本没有引出来，而一根防拆线就能把三个核与两个缓冲区 一起放倒。软件拿到的恰好是算法定义说它该拿到的那些字节。（`dk` 究竟该不该 返回，见 [SECURITY.zh-CN.md](#)。）

pqc_accel_axi — 仿真与主机路径

这是 `include/pqchsm/accel.h` 背后的从机，由主机软件栈驱动，而不是由板上程序 驱动。控制面走 AXI4-Lite，数据面走 AXI4-Stream。译码用 `[4:2]`，所以 `0x20` 及以上的地址会回绕成别名。

偏移	名称	访问	说明
0x00	CTRL	W	[0] START, [1] SOFT_RESET
0x04	STATUS	R	[0] DONE, [1] BUSY, [2] ERR
0x08	MODE	RW	操作码
0x0C	PARAM	RW	参数集
0x10	IN_LEN	RW	输入长度，单位字节
0x14	OUT_LEN	R	输出长度，由硬件写入
0x18	ERRCODE	R	失败详情，由硬件写入
0x1C	VERSION	R	常量 0x0001_0000

编码	操作	IN_LEN	OUT_LEN
7	NTT, 正变换	512	512
8	NTT, 逆变换	512	512
9	Keccak-f[1600]	200	200
10	SHAKE / SHA3, 完整海绵	0...512	1...512

编码 10 是唯一使用 `PARAM` 的，里面打包了三个字段：`[7:0]` 域后缀（`0x1F` 为 SHAKE，`0x06` 为 SHA3），`[15:8]` 以字节计的 rate（168/136/72，8 的倍数），`[31:16]` 请求的输出长度。把输出长度放在这里而不是另加一个寄存器，是为了保持 寄存器表不变。

编码 9 与 10 共用一个 `keccak_f1600`：模式 10 驱动 `sha3_core` 里的海绵，模式 9 则经一个只在海绵空闲时才有效的直通端口借用底下的置换。同一时刻只有一条 命令在跑，所以不存在争用——而再放一个置换核大约要花 3500 个 LUT。模式 10 在 报告 `DONE` 之前会清掉海绵，因为挤出没有自然的终点（SHAKE 的输出长度是任意的），否则海绵永远回不到空闲，也就永远不再把置换借出去。

其他任何编码，或与编码不匹配的长度，都会以 `ERR` 和 `ERRCODE = 3`（“模式未 实现”）结束。报告失败是有意的：悄悄退回到另一种实现，会让一个只实现了一部分 的加速器看起来是完整的。

数据面。位宽 32，缓冲区按字寻址，小端以匹配软件。每个输入包都从缓冲区 偏移 0 开始写；带 `TLAST` 的那一拍被接受之后，写指针回到 0，所以软件不需要“复位写指针”这样的寄存器。`BUSY` 置位期间 `TREADY` 为低。完

成后，**OUT_LEN** 换算成字之后可读；拉高 **TREADY** 即可把结果抽走，最后一拍带 **TLAST**，这会复位 读指针并撤下 **TVALID**——一个结果恰好只能取走一次。

```
stream input packet (TLAST on the last beat)
write MODE, IN_LEN
write CTRL = START
poll STATUS until DONE
if ERR: read ERRCODE
else: read OUT_LEN, then drain the output stream
```

通用约定

START 自清。写 1 启动一条命令，硬件在同一周期清掉该位，所以 **CTRL** 回读 永远是 0。没有这一条，软件对 **CTRL** 的任何读-改-写都会把命令再触发一遍。

DONE 是锁存电平，不是脉冲。它在完成时置位，一直保持到下一次 **START**。软件 在任意时刻轮询，单周期脉冲会被错过。清除绑定在 **START** 上，而不是绑在对 **STATUS** 的读上，这样读寄存器就绝不会毁掉它所报告的状态。

ERR 与 **DONE** 同时出现，在同一周期，详情在 **ERRCODE** 里。没有另外一条需要 轮询的失败通路。

BUSY 反映数据通路，不是一个锁存位：从 **START** 起一直高到完成。

状态、长度与错误寄存器由硬件写入，对软件只读。写入被忽略，并以 **OKAY** 而非 **SLVERR** 应答：在 AXI4-Lite 上，对只读寄存器的写是无害的，而错误应答会 把不了解这套约定的主设备卡死在那里。

被拒的访问没有副作用。特别是被拒的读绝不弹出 **TRNG** 的 **FIFO**——否则一次拿 不到数据的非安全读仍然能抽干熵池。

是 **RAZ/WI**，不是 **DECERR** —— 这一条以前写的正好相反。原文是：“是 **DECERR**，绝不是 **OKAY** 加返回零：悄悄返回零会让普通世界以为自己读到了东西。”那个顾虑 是真的，现在也没消失，只是被更大的问题压过去了：**DECERR** 的 **posted** 写以 **SError** 回来会打穿内核，于是按旧规则，任何一个打错的地址都要付一次断电。“崩不了板”赢了这一局。

旧顾虑靠三条压住：

- **STATUS** 也读回 0，于是 **ENABLED** 与 **READY** 都是 0 —— 任何一个先查就绪 再取数的消费者（本来就该这么写）立刻看到“没就绪”，根本走不到那些零；
- **VERSION** 读回 0，而它在每个核上都是非零常量。这是现有最响亮的“你被拒了”信号，边界的反向证明现在就用这一条；
- 违规计数器记下这一笔，且只有安全世界读得到。

真正丢掉的是这个：一个既不查就绪也不查版本、闷头读 **RDATA** 的消费者，现在会 拿到全零而不是一个错误。这样的消费者本来就是坏的（它对真实的 **underrun** 也一样 会拿到坏数），但失败方式确实从“总线替你挡住”变成了“你自己得写对”。这是一次实实在在的取舍，写在这里，不藏在别处。

安全模型

边界是什么、在设备上证明了什么，以及——用同样的篇幅——什么没有证明。上报漏洞见 [../SECURITY.md](#)。

这是一个研究原型。这里没有任何东西通过认证，它不应被用来保护任何真实的东西。

- [边界](#)
- [威胁模型](#)

- [已经证明了什么](#)
- [哪些没有防护](#)
- [局限](#)

边界

密码边界是硬件。 ML-KEM 核、对称核（AES-128/256、SM4、SM3）、环形振荡器熵源、密钥仓与访问门控全部位于 FPGA fabric 之中。主机软件——密钥库、槽位元数据、包裹、PKCS#11——跑在它之外。

边界由三个机制构成：

1. **axi4lite_firewall** 按 **AxPROT[1]** 对每个从机做门控。以 **SECURE_ONLY=1** 构建的从机只接受安全事务，其余一律读回 0、写被丢弃，且不产生任何总线错误（RAZ/WI）。
2. **axi4lite_xbar** 做完全译码。孔径高位、槽位号、槽内偏移高位与 32 位对齐 必须全部成立，否则事务就地被拒——同样是 RAZ/WI。不存在镜像地址。
3. **key_vault** 没有读路径。密钥经总线写入，只通过私有的 **use_key** 线出来，进入密码核。这是 RTL 的结构性质，不是一次权限检查。

PL 防火墙是这条路径上**唯一**的强制层。UG1085 v2.5 对此有定论：**0x8000_0000** 不出现在任何 XPPU 孔径中（表 16-10），而 FPD_XMPU 不在 **M_AXI_HPM0_LPD** 的路径上——到达 PL 是“不经过 FPD”的（p. 1092）。PS 侧的保护单元根本覆盖不到 PL 窗口。这不是本设计的缺口；它正是译码必须精确的原因。

（顺带实测：这块板上 PS 侧的保护单元完全未配置——XPPU **CTRL = 0**，全部 400 条权限项停在复位默认值，XMPU_OCM 各区域全部禁用，且 XPPU 未加锁。这与上文无关：即便完全配置好，它们也够不到 PL。）

威胁模型

防护的对象。 拥有普通世界（EL1-NS）代码执行权限、包括 root 在内、想把密钥材料从模块中读出来的攻击者。他们可以经 **/dev/mem** 发起任意 AXI 事务，可以扫遍整个窗口；他们无法从密钥仓中恢复出对称密钥，也无法读取任何以 **SECURE_ONLY=1** 构建的从机。

不防护的对象。

- 拥有**安全世界（EL3）代码执行权限**的攻击者。EL3 是这里的信任根；攻破 BL31 就攻破了一切。
- 能**重配 PL**的攻击者。装入另一份 bitstream 就把边界换掉了。这块板上没有任何东西能阻止它，因为安全启动没有开通（见[局限](#)）。
- 拥有**物理接触**的攻击者。没有机壳，没有接上的防拆响应，没有环境防护。RTL 里有一个能清零密钥仓的 **tamper** 输入；没有任何东西驱动它。
- 测量**功耗或电磁辐射**的攻击者。刻意排除在范围之外——见下文。
- 能**读取主机进程内存**的攻击者。主机栈在边界之外：明文密钥材料在那边的进程内存中是存在的。缓冲区在可能之处被清零并 **mlock**，仅此而已。

root 仍然能驱动硬件。 EL3 SiP 目前暴露的是白名单内的 MMIO 读与写，操作序列由普通世界的守护进程拼装。所以普通世界读不到密钥材料，但 root 仍然能装载、擦除和使用密钥仓槽位。要堵上这一点，需要一个操作粒度的 SiP（“用句柄 H 解封装这段密文”，而不是“往这个地址写这个字”），也就意味着一个跑在 EL3 里的带缓冲状态机——而 EL3 没有异常处理，一次取指出错就是一个核当场卡死。它需要自成一套的验证方法，本项目没有尝试。

已经证明了什么

全部在真实设备上。

门控双向生效。安全世界 (EL3, `AXPROT[1] = 0`) 读一个 `SECURE_ONLY=1` 的核, 得到 `VERSION = 0x00010000`。普通世界 (EL1-NS, `AXPROT[1] = 1`) 读同一个地址被拒——而/同一个普通世界能成功读到一个 `SECURE_ONLY=0` 的核。两半都需要: 第二半才排除掉”这个地址本来就不可达”。

“被拒”的观测方式在这次测量之后改了。上面这次运行早于 RAZ/WI 改动, 记录到的是 DECERR/SIGBUS。防火墙现在以数据 0、不产生总线错误的方式拒绝, 所以在当前位流上同一个实验会在放着 `0x00010000` 的地址上读到 0。

新判据是两者中更强的一条, 这也是采用它而不只是容忍它的原因: DECERR 区分不了”门关着”和”这个地址根本没东西”——空地址回的也是 DECERR。而在一个放着非零常量的地址上读到 0, 同时证明了两件事: 事务确实到达了那个从机, 并且值被扣下了。

在 RAZ/WI 位流上重跑这次双向测量还欠着。新观测量目前由仿真覆盖。

对称密钥不越过边界。在两个从机上各扫了 256 字节——48 个读得到, 80 个被防火墙拒绝。密钥的四个字一个都没在任何地方出现, 而用这些密钥算出的密文对照 GB/T 32907 逐字节一致。这里同样两半都需要: “什么都没找到”和”密码算得对”单独都不能证明任何事。

算法正确性锚定在公开向量上, 而不是本项目自己的模型上。ML-KEM 512/768/1024 的 KeyGen/Encaps/Decaps 在真硅上与 NIST ACVP 逐字节一致 (20/20); AES 对照 FIPS 197, SM4 对照 GB/T 32907, SM3 对照 GB/T 32905。

熵是测出来的, 不是假设出来的。导出 1,048,576 个调理前样本, 以 SP 800-90B 非 IID 估计器评估: $H = 0.871234$ bit/sample。这次测量随后确定了健康测试的判决门限。也只能如此: 先前的门限是在假设 $H = 0.5$ 之下算出来的, 给出的 APT 触发概率是 5.5×10^{-52} ——这个测试根本不可能触发, 所以此前每一条”无 APT 告警”的记录都一文不值。见 [TESTING.zh-CN.md](#)。

隐式拒绝路径上的时序是平的。ML-KEM Decaps, 一边是合法密文, 另一边是翻转了单个比特的密文, 各跑 200 次: 中位延迟相差 0.000 %。真正的提前退出会表现为数量级的差异, 而不是几个百分点。

以上全部的原始日志逐字保存在 [../board/logs/](#)。

哪些没有防护

功耗与电磁侧信道刻意排除在范围之外, 理由有三条, 写出来是为了让这读起来是一个决定, 而不是一处遗漏:

1. 验证一项对策需要侧信道实验台——示波器、功耗与电磁探头、TVLA 方法学。没有它, 实现了掩码却无法证明它有效, 等于把”看起来做完了”当成”做完了”, 这比不做更糟。
2. 一阶掩码把每一个中间值拆成份额, 每一份都需要随机数。放到 ML-KEM 数据通路上, 这极可能超出 ZU3EG 的 71K LUT——已经用掉 50.5 %——从而被迫换更大的器件。
3. 时序与功耗/电磁是不同的攻击面。常量时间那部分工作覆盖前者, 不能替代后者。

对 liboqs 与 OpenSSL 不作任何主张。主机软件路径上的每一个 ML-KEM 与 ML-DSA 操作都跑在 liboqs 内部。那里的一个时序侧信道就是这里的一个时序侧信道, 而本仓库中没有任何检查能发现它。

常量时间审计是词法级的, 不是编译器级的。它看不见编译器引入的分支、编译器拒绝发出的 `cmov`, 也看不见被溢出到栈槽后又重新载入的秘密。经验性的时序测试覆盖 了比较原语; 源码的其余部分只被扫描覆盖。

对进程地址空间之外的内存不作任何主张——CPU 寄存器、被换出的页面，或断电之后的 DRAM 内容。`mlock` 是尽力而为，且未在内存压力下验证过。

局限

下面每一项都足以单独卡住一次认证送检。列出它们，是为了让本文不被误当成一份就绪 声明。

ML-KEM 私钥会离开硬件	<code>KeyGen</code> 经 AXI 返回 <code>ek dk</code> ，因为 ACVP 校验需要如此。守护进程持有 <code>dk</code> ，只给应用一个句柄，所以它不越过接口——但”私钥永不离开硬件”是假的，本文也不作此主张。要修好它，就得在 <code>mlkem_axi</code> 内部加上私钥存储与基于句柄的使用
没有设备绑定	密钥派生根（ <code>src/crypto/kdr.c</code> ）是一个编译进去的常量，其字面文本自己就这么写着。真正的设备会从 eFUSE、BBRAM 或 PUF 取它。eFUSE 不可逆，而板子只有一块；BBRAM 需要 JTAG
没有安全启动	BBRAM 密钥开通需要 JTAG。PL 配置是运行时装入的，不是来自一份签名的黄金镜像
没有模块完整性检查	使用之前没有任何东西校验模块镜像
没有物理安全	只有逻辑层面的强制。 <code>tamper</code> 输入在 RTL 中存在并会清零密钥仓；没有任何东西接到它上面
缺少条件自测	对生成的密钥对没有成对一致性测试，也没有连续随机数发生器测试
没有 SM2	SM4 与 SM3 已实现；SM2 非对称算法没有
ML-DSA 只有算子	13 个模块已对照参考模型验证，未串成 KeyGen/Sign/Verify
没有算法证书	ACVP 向量是在本地跑的。那是正确性的证据，不是证书
审计日志单写者	审计模块不对文件加锁
Shamir 份额校验和无密钥	它们检测的是损坏，不是篡改
SO PIN 锁定未强制	失败次数有计数，但槽位不会被锁定，因为锁上就等于把设备变砖；恢复走 M-of-N
<code>CKM_HASH_ML_DSA_*</code> 未实现	PKCS#11 多段签名把整条消息缓冲起来，而不是流式地做摘要
主机 RNG 是 OpenSSL	PL 熵源在边界之内，但主机软件仍然经 OpenSSL 从操作系统取种子
保持时序余量很薄	经显式管理之后，有效保持余量为 0.110 ns。保持违例不能靠放慢时钟解决，所以 RTL 一改动就必须重新核对这个数字；实现流程在低于 0.050 ns 底线时中止

有两项卡在 JTAG 硬件上，而不是卡在设计工作上：把 PL 配置固化进黄金 `BOOT.BIN`，以及 BBRAM 支撑的安全启动。有两项在这块板上被永久排除：烧 eFUSE（不可逆，只有一块板）与 PUF 黑密钥（依赖于前者）。

一份 FIPS 140-3 / GM/T 0028 风格的安全策略草稿，带有它自己的差距清单，见 [reference/security-policy.zh-CN.md](#)。

测试

检查了什么、用什么手段检查，以及本仓库中引用的每一个数字如何复现。

- [概览](#)
- [原则](#)
- [RTL 验证](#)
- [实现流程断言](#)
- [真硅结果](#)
- [熵](#)
- [主机软件测试](#)
- [复现](#)

概览

检查项	结果	位置
cocotb RTL 回归	200 个测试，0 失败	<code>./tools/rtl_sim.sh</code>
Verilator <code>-Wall</code> + Icarus lint	70 个模块，0 警告	<code>./tools/rtl_lint.sh</code>
Yosys 可综合性	68 个模块，全部可综合	<code>./tools/rtl_synth_check.sh</code>
ML-KEM 512/768/1024 对照 NIST ACVP，真硅上	20 / 20 逐字节一致	<code>board/logs/RESULT_seckem3.txt</code>
板级自测	24 / 24	<code>board/logs/RESULT_hwtest.txt</code>
审计后板级重跑	PASS 25 / FAIL 0 / SKIP 6	<code>board/logs/RESULT_audit.txt</code>
边界反证	48 个读得到的地址中 0 个密钥字	<code>board/logs/RESULT_hwtest.txt</code>
AxPROT 门控，双向	闭合	<code>board/logs/RESULT_secproof.txt</code> 、 <code>RESULT_secneg.txt</code>
TRNG 最小熵	$H = 0.871234 \text{ bit/sample}$	<code>tools/sp800_90b.py</code>
SP 800-90B 重启测试	$H_{\text{restart}} = 0.745427$ ，通过	<code>board/logs/RESULT_restart.txt</code>
Decaps 时序，合法 vs 隐式拒绝	中位差 0.000 %	<code>board/logs/RESULT_seckem3.txt</code>
经 SDF 接口的端到端	通过，另加 6/6 反证（测于 RAZ/WI 之前，当时记录为 DECERR）	<code>board/logs/RESULT_service.txt</code>
<code>ctest</code> （主机软件）	46 / 46，4109 条断言	<code>ctest --test-dir build</code>
ASan + UBSan / TSan / <code>leaks</code>	全过 · 0 竞争 · 0 泄漏	<code>docs/USAGE.zh-CN.md</code>
libFuzzer	138 万次执行，无崩溃	<code>./tools/fuzz.sh</code>

检查项	结果	位置
aarch64 Linux (GCC 12)	全过	<code>./tools/aarch64_test.sh</code>
RAZ/WI 上板复验	还没跑 —— 位流已出、仿真已过，板子够不到 (SSH key 没扛过断电)	<code>board/src/hsm_nocrash.c</code> 、 <code>hsm_secneg.c</code>

原则

有三个习惯贯穿本仓库中的每一项测试。

独立的判据。 一个结果不会因为它与本项目自己的模型对得上就被信任。Keccak 对照 公开的全零置换向量以及 `hashlib` / OpenSSL 检查。NTT 对照一个从不碰旋转因子表的 教科书式负循环卷积检查，并通过重建 ML-KEM 密钥生成、逐字节复现 ACVP 的 `ek` / `dk` 来检查。KMAC 对照 NIST 文档、OpenSSL 以及另一份独立的 Keccak 检查。

结构性检查优于习惯。 那些功能测试观察不到的性质——密钥派生根没有读回接口、`src/` 中没有依赖秘密的分支或索引、每一个密钥材料字段都被其析构函数擦除——被写成 扫描器并接入 `ctest`。每个扫描器**先在合成样本上自测**：一个什么都找不到的扫描器，在报告“干净”时什么也证明不了。

阴性对照。 断言要靠弄坏一样东西、确认测试确实失败来验证——扰动一个旋转因子、去掉一层 NTT、翻转 Keccak 轮常数中的一个比特、在 TSan 下移除一把锁、加入一个假的 密钥读回函数、对一个故意提前返回的比较计时、探测一个从未被擦除的栈帧。

同样的纪律也管着结果该怎么读。边界反证在两个从机上各扫 256 字节，**同时**报告 48 个读得到的地址中没有出现任何密钥字，**以及**这些密钥算出的密文是正确的。任何一半单独拿出来都证明不了什么：沉默也可能意味着密钥压根没被装载。

RTL 验证

跨 26 个顶层的 200 个 cocotb 测试，在 Icarus Verilog 下运行：

组	测试
ML-KEM 算子与数据通路	<code>mont_reduce</code> 、 <code>butterfly</code> 、 <code>basemul</code> 、 <code>ntt_core</code> 、 <code>compress</code> / <code>decompress</code> (5 种位宽)、 <code>sample</code> 、 <code>bitpack</code> ，以及完整的 KeyGen/Encaps/Decaps
ML-DSA 算子	<code>tb_mldsa_units</code> (6)、 <code>mldsa_ntt_core</code> (5)
Keccak	<code>keccak_f1600</code> (5)、 <code>sha3_core</code> (9)
总线	<code>axi4lite_xbar</code> (6)、 <code>axi4lite_firewall</code> (6)、 <code>pqc_accel_axi</code> (16)、 <code>key_vault</code> (4)、 <code>key_vault_axi</code> (6)、 <code>mlkem_axi</code> (9)
对称与商密	<code>aes_core</code> (5)、 <code>sm4_core</code> (6)、 <code>sm3_core</code> (6)、 <code>sym_vault_top</code> (5)
风扇	<code>fan_ctrl</code> (7)、 <code>sysmon_drp</code> (3)

组	测试
TRNG	<code>trng_health</code> (8)、 <code>trng_source</code> (3)、 <code>trng_cond</code> (4)、 <code>trng_top</code> (4 + 2 不丢弃 + 1 丢弃)、 <code>trng_axi</code> (6)、告警路径 (4)、原始抽头 (3)

Lint 把每一个模块都当作自己的顶层来跑，而不只是跑集成后的顶层：否则那些没有被 任何地方例化的模块——组合算子、采样器——永远不会被展开，也永远不会被检查。真正 出于设计意图的告警在 `hardware/rtl/lint_waivers.vlt` 中逐条豁免，按文件名与信号名 匹配，这样同类的新告警不会被一并吞掉。用 Icarus 再跑一遍能抓到另一类问题；两个 仿真器在位宽截断语义上的差异本身就是一次交叉检查。

实现流程断言

`hardware/syn/impl_bitstream.tcl` 在综合之后立即检查并在失败时中止，这样缺陷就不会 拖到三十五分钟之后才被发现，更不会拖到板子上。

断言	学到它的代价
PS 的 <code>acclk</code> / <code>rlast</code> / <code>bid</code> / <code>rid</code> 必须被驱动	两次板子挂死，两次断电重启
<code>fan</code> 必须落在 <code>PACKAGE_PIN AA11</code> / LVCMOS33 上	风扇引脚接错在运行时没有任何症状；它只是悄悄地把芯片烤热
<code>SYSMONE4 SIM_DEVICE = ZYNQ_ULTRASCAL</code>	一次跑满 35 分钟的实现在最后的 DRC 上失败
WNS < 0 就不出 bitstream	一份不收敛的 bitstream 在板上产生的失败看起来像算法 bug，而且追查代价极高
有效保持余量 ≥ 0.050 ns	三份 bitstream 停在刀锋边上（见下文）

保持余量底线值得单独记一笔。三份 bitstream 实测的 WHS 分别是 +0.001 / +0.010 / +0.013 ns——全为正，全部被 Vivado 签核通过，全都实质上等于零，并且随 RTL 漂移。保持违例不能靠降低时钟频率来修。做法是把要求抬高，让布线器去 满足它：

```
set_clock_uncertainty -hold 0.100 [get_clocks -of_objects [get_pins u_div/0]]
```

布线过程中 WHS 从 -0.150 → -0.027 → +0.010，买到大约 0.16 ns 的真实余量，代价是 在可用的 3.5 ns 中花掉 0.14 ns 的建立余量。有效保持余量从 0.010 ns 变成 **0.110 ns**。这条底线断言在第一次运行时就抓住了先前那三份 bitstream；没有它， 它们每一份都会悄无声息地发出去。

真硅结果

算法。ML-KEM 512/768/1024 的 KeyGen/Encaps/Decaps 对照 NIST ACVP 的 prompt/expectedResults 配对：20/20 逐字节一致。长度由 RTL 从 `pset` 字段推导，所以软件报不出一个错的长度。AES-128/256 对照 FIPS 197 C.1/C.3，SM4 对照 GB/T 32907 A.1，SM3 对照 GB/T 32905 A.1。

审计后复验（`RESULT_audit.txt`），在改动了译码、擦除状态机与 TRNG 采样 FIFO 之后：

- 地址混叠，13/13——五次正向读成功；八个镜像地址（+0x110、+0x100、+0x1100、+0xFF00、+0x8000、孔径之外、槽位 6、槽位 7）全部被拒。

这次测量早于 RAZ/WI 改动，所以它当时记录到的是那八个地址回 DECERR/SIGBUS。用例断言的”被拒”本身没变，变的只是观测量（现在是读回 0）。在 RAZ/WI 位流上重跑这一条还欠着 —— 板子的 SSH key 没扛过上次断电。新观测量由仿真覆盖（test_xbar 8/8，含 64 KB 全扫）。

- 清零，4/4——STATUS.WIPING 置起，持续 112.5 μs，对照理论值 109.2 μs（8192 个周期 @ 75 MHz；差值是轮询开销），之后 OUT_LEN = IN_PTR = 0，且相同 输入重现出 2432 个完全一致的字节。
- TRNG，3/3——在无人读取期间吸收了 BLOCKS = 17892 而 DROPS = 0。修复之前 这个计数器读出来是 65535，已经饱和。
- 非法参数，4/4——mode=3、pset=3，以及两者同时：PARAM_ERR 置位，BUSY 始终不抬起，先前的 DONE / OUT_LEN 被作废，之后一组合法参数能正常跑完。

算法回归在同一轮里跑了，结果没有变化，24/24——改了三个子系统，输出字节一个都 没变。

吞吐 @ 75 MHz，每个操作 20 次：

参数集	KeyGen	Encaps	Decaps
ML-KEM-512	1.08 ms (924/s)	0.75 ms (1339/s)	0.98 ms (1018/s)
ML-KEM-768	1.65 ms (605/s)	1.12 ms (895/s)	1.44 ms (694/s)
ML-KEM-1024	2.27 ms (440/s)	1.57 ms (636/s)	1.96 ms (510/s)

这些数字包含逐字节的软件 AXI 传输，不是硬件核的耗时。把它们发布出来是为了那个 比值——跨参数集大致 1 : 1.5 : 2.1，与 k = 2/3/4 的工作量吻合——那才是有意义的部分。

资源与时序，在 xazu3eg-sfvc784-1-i 上完整布局布线：

CLB LUT	35,659 / 70,560 (50.54 %)
寄存器	25,977 (18.41 %)
Block RAM	15.5 / 216 (7.18 %)
DSP	140 / 360 (38.89 %)
外部引脚	1 (风扇, AA11)
WNS / 有效保持余量	+3.325 ns / +0.110 ns @ 75 MHz

RAZ/WI 那次改动的增量：+48 LUT、+61 寄存器，DSP 与 BRAM 不变。那是四个新增的 16 位饱和计数器（译码器读/写、TRNG 读/写）连同它们的寄存器读出通路 —— RAZ/WI 的 响应本身其实比 DECERR 更省，它把一个响应码多路选择器换成了一个常量。有效保持余量没变，仍是 +0.110 ns；建立余量从 +3.504 降到 +3.325 ns，对 13.3 ns 的周期仍有 3.3 ns 富余。

这套流程在构建机上是确定性的：lint 清理（位宽修正与死代码删除）前后跑出来的资源 与时序结果逐比特一致，这本来就是应该的——Vivado 本来就会删死代码，而位宽修正不会 改变所实现的逻辑。

熵

用一份表征用 `bitstream` (`RAW_TAP=1`；在生产构建里这条路径根本不存在，而不是返回 零) 导出了 1,048,576 个 **调理前** 样本。取调理前是必须的：SHA-3 海绵的输出无论进去 的熵有多少都看起来是均匀的，所以去评估 **RDATA** 只会得到一个漂亮而无意义的数字。

```
$ python3 tools/sp800_90b.py /tmp/trng_bits.bin
MCV 0.996191 · Collision 0.871234 · Markov 0.996108 · Compression 1.000000
t-Tuple 0.923437 · LRS 0.993807 · MultiMCW 0.999567 · Lag 0.987625
MultiMMC 0.994768 · LZ78Y 0.993032
min-entropy (minimum of the ten) 0.871234
```

配套统计：1 的比例 0.500064，最长游程 19 (100 万比特下期望 ≈ 20)，滞后 1–8 的 自相关全部低于 0.4 %，32 位字内无位置偏置，采集期间无健康告警。

这个工具不是 NIST 的参考实现。构建机没有外网，也没有 NIST EntropyAssessment 的副本。

`tools/sp800_90b.py` 直接实现了 SP 800–90B (2018–01 终稿) §6.3.1–6.3.10，其可信度建立在能复现规范中每一节的算例上：`python3 tools/sp800_90b.py --selftest` 复现其中九个，精确到小数点后四位。第十个 LZ78Y 在规范中没有算例，且它与另外四个预测器共用已被验证的尾部计算。

这次测量在两个不同方向上作废了先前的门限：

参数	旧取值在实测 H 下实际意味着什么	后果
RCT = 41	在约 9.4 M samples/s 下 $\alpha \approx 2^{-34.8}$	每 55 分钟一次误报——对一个常开的源来说太频繁
APT = 793	触发概率 5.5×10^{-52}	这个测试根本不可能触发。此前每一条“无 APT 告警”的记录都毫无价值

按 $\alpha = 2^{-40}$ 重算 (在这个采样率下大约每 33 小时一次误报)：**RCT 41 $\rightarrow$ 47**，**APT 793 $\rightarrow$ 672**。 α 不是照抄规范里的 2^{-20} ，那个值在 9.4 M samples/s 下意味着每 0.11 s 一次误报—— **α 必须按采样率来选**。作为对推导过程的 校验，这个公式先在 $H = 0.5$ 下重新求值，精确重现了 RTL 里已有的 41/793，然后才被 用来往前推。

采集是**有间隙的**：抽头 FIFO 深 64 个字，满了就丢弃新样本，而不是对噪声源施加 反压，因为反压会改变正在被评估的那条流本身。这不影响最小熵估计，但重启测试的 数据必须单独采集 (已经采了； $H_{\text{restart}} = 0.745427$ ，通过)。

主机软件测试

46 个 **ctest** 目标，4109 条断言：槽位状态机与并发、密钥库与包裹、备份/Shamir/注入、审计链与锚定、PKCS#11、KAT 解析、常量时间计时、清零，以及各个加速器 transport ——包括”软件桩与 Verilator 仿真的 RTL 逐字节一致”这条断言。

390 条 NIST ACVP 向量在软件中做逐字节校验，其中 60 条被显式跳过并如实报告，而不是 悄悄地算作通过。

消毒器与平台运行：ASan + UBSan 全过；ThreadSanitizer 0 竞争 (通过移除锁来验证，那时报出 9 条)；macOS **leaks** 干净；libFuzzer 138 万次执行无崩溃；aarch64 Linux 在 GCC 12 下全过。

复现

本节中没有任何一项需要板子。

```
./tools/rtl_sim.sh           # 200 个 cocotb 测试
./tools/rtl_lint.sh          # Verilator -Wall + Icarus
./tools/rtl_synth_check.sh   # Yosys 可综合性
python3 tools/sp800_90b.py --selftest # 复现规范中的算例
python3 tools/ct_audit.py --self-test # 常量时间扫描器，先跑对照
python3 tools/check_zeroize.py --self-test

./tools/fetch_vectors.sh && cmake -S . -B build && cmake --build build -j
ctest --test-dir build --output-on-failure
```

生成 bitstream，需要一台装有 Vivado 2020.1 的机器（约 35 分钟）：

```
vivado -mode batch -source hardware/syn/impl_bitstream.tcl
# → hardware/syn/impl/zu3eg_hsm.bit
PQC_CHARACTERIZE=1 vivado -mode batch -source hardware/syn/impl_bitstream.tcl
# → hardware/syn/impl/zu3eg_hsm_char.bit (低风扇门限 + TRNG 原始抽头；
# 这是表征用的构建，不是产品形态)
```

在板子上，每一个碰到 PL 的动作都必须走 harness——见 [USAGE.zh-CN.md](#)。